

Artifact: A Reproducible, Database-Free Pipeline for Detecting Research-Software Paradigms in the DELFI Community

Julian Dehne ¹, Fabian Lochner², and Axel Wiepke¹

Abstract: This artifact accompanies the DELFI 2026 paper *Research Software Engineering in the Learning Technologies: a replication study of research paradigms in the DELFI community*. It provides a self-contained, file-based pipeline that reproduces the study’s label aggregation, inter-coder reliability (ICR) metrics, descriptive figures and secondary bias analysis directly from the shipped intermediate data — *without* access to the copyrighted DELFI paper PDFs or to a commercial LLM API. The six pipeline steps are orchestrated by a single management script; the two steps that require the paper texts (preprocessing, human re-annotation) and the one that requires an LLM API token (the experiments) are skipped by default. A programmatic guard enforces that no paper full text is ever contained in the released data. We describe the artifact’s structure, how to run it, the reproducibility guarantees, and the constraints that motivated its design for the artifact-evaluation track.

Keywords: Research Software Engineering, replication, large language models, inter-coder reliability, reproducibility, DELFI

1 Introduction

The companion study [DLW26a] replicates and extends a 2017 analysis of research-software paradigms across the DELFI proceedings, using three open-weight large language models (LLMs) to annotate $n = 798$ papers and validating the result against the earlier human-coded reference study. This artifact packages the *evidence and the computation* behind those results so that a reviewer can re-run the analysis end-to-end on a laptop in minutes.

The central design constraint is legal: the analyzed DELFI papers are copyrighted and may not be redistributed. The artifact therefore ships only derived data (LLM labels, justifications, human re-annotations and bibliographic metadata) and is explicitly built so that the two paper-dependent steps and the API-dependent step can be skipped while still reproducing every published number and figure.

2 Artifact overview

The artifact is a Python package (pipeline/) plus shipped intermediate data (data/) and this paper (artifact-paper/). It is database-free: whereas the original study stored papers in

¹ University of Potsdam, An der Bahn 2, 14476 Potsdam, Germany, dehne3@uni-potsdam.de,  <https://orcid.org/0000-0001-9265-9619>; axel.wiepke@uni-potsdam.de

² Affiliation of the second author, fabian.lochner@example.org

MySQL, every step here reads and writes CSV files under `data/`. We claim the *Available* and *Reusable* properties: the package is permanently archived, documented, and parameterised so that it can also be re-run on new inputs.

3 Repository structure

```
rse_in_delfi_artifact/
  run_pipeline.py      # management script (skip-aware)
  pipeline/            # config + the six steps + full-text guard
  data/                # shipped intermediate products (no full texts)
  artifact-paper/      # this LNI document
```

4 The pipeline

The pipeline has six steps (Table 1). Steps 3, 5 and 6 run from the shipped data and constitute the default reviewer run; steps 1, 2 and 4 are *skipped by default* because they need resources reviewers do not have.

Tab. 1: Pipeline steps and default execution policy.

#	Step	Default	Requires
1	Preprocessing (PDF $\rightarrow$ text)	skipped	paper PDF folder
2	LLM experiments	skipped	SAIA/KISSKI API token
3	Descriptive analysis (figures)	runs	shipped data
4	Human re-annotation	skipped	paper PDF folder (interactive)
5	Label aggregation (intra + inter, ICR)	runs	shipped data
6	Secondary analysis (paradigm biases)	runs	shipped data

Data flow. Step 1 extracts text from the paper PDFs; step 2 sends each paper to three LLMs (three runs each) via the KISSKI SAIA API and stores the labels and justifications. Step 5a aggregates the three runs per model (majority vote) and computes intra-model ICR; step 4 elicits a human label for the papers on which all three models disagree; step 5b aggregates across models (majority vote, ties broken by the human label), reports cross-model ICR and validates against the 2017 reference study. Steps 3 and 6 produce the figures and the secondary bias analysis from the final aggregated labels.

Management script. `run_pipeline.py` runs the default steps, accepts `--paper-folder` as a parameter to enable steps 1 and 4, and `--run-experiments` (with a SAIA token in the environment) to enable step 2. A no-full-text guard runs before and after every invocation.

5 Reproducing the results

```
python -m pip install -r requirements.txt
python run_pipeline.py          # steps 3, 5, 6 from shipped data
```

This regenerates `data/inter_LLM/df_llm_experiments_final_aggregated_*.csv` (798×185), the ICR tables, the paradigm-distribution and label-trend figures, and the combined-label bias tables.

6 Reproduced results

Table 2 shows the cross-model ICR recomputed by step 5 on the shipped data, reported as Krippendorff’s α [Kr04] (and, for binary labels, complemented by Gwet’s AC1 [Gw08] in the released tables). All three models disagreed completely on 0, 6, 17 and 9 papers for the process, outcome, design and acceptance paradigms respectively; those cases were resolved with the shipped human labels.

Tab. 2: Cross-model inter-coder reliability (Krippendorff’s α), recomputed.

Label group	Krippendorff’s α
Binary labels (average)	0.51
Ordinal labels (average)	0.66
All labels (average)	0.60
Reference-study validation (binary, average)	0.33

7 Data and the no-full-text guarantee

The released data contain only labels, LLM/human justifications and bibliographic metadata. The original annotation script dropped the `abstract`, `text` and `references` columns before persisting results “for legal reasons”; the artifact preserves this invariant and enforces it programmatically. `pipeline/fulltext_guard.py` scans every shipped table and fails if a banned full-text column is present or if any cell exceeds a length threshold incompatible with a short justification. Step 1’s full-text output is written only to a `.gitignore`’d directory.

8 Requirements and environment

Python 3.11+, `pandas`, `numpy`, `scikit-learn`, `krippendorff`, `statsmodels`, `matplotlib` and `openpyxl`; see `requirements.txt`. `pymupdf` is needed only to re-run step 1, and an OpenAI-compatible client plus a SAIA token only to re-run step 2.

9 Limitations

Steps 1 and 2 cannot be exercised by reviewers without the copyrighted PDFs and an API token; their outputs are shipped and verified downstream instead. LLM annotations are sampled at temperature 0 with a fixed seed, but exact reproducibility of step 2 ultimately depends on the provider’s model snapshots.

References

- [DLW26a] Dehne, J.; Lochner, F.; Wiepke, A.: Research Software Engineering in the Learning Technologies: a replication study of research paradigms in the DELFI community. In: DELFI 2026 – Die 24. Fachtagung Bildungstechnologien der Gesellschaft für Informatik (GI). Lecture Notes in Informatics (LNI), Gesellschaft für Informatik, Bonn, 2026.
- [DLW26b] Dehne, J.; Lochner, F.; Wiepke, A.: rse_in_delfi_artifact: A reproducible, database-free pipeline for detecting research-software paradigms in the DELFI community, https://github.com/juliandehne/rse_in_delfi_artifact, Replication artifact., 2026.
- [Gw08] Gwet, K. L.: Computing Inter-Rater Reliability and Its Variance in the Presence of High Agreement. *British Journal of Mathematical and Statistical Psychology* 61 (1), pp. 29–48, 2008.
- [Kr04] Krippendorff, K.: Reliability in Content Analysis: Some Common Misconceptions and Recommendations. *Human Communication Research* 30 (3), pp. 411–433, 2004.